

Assembly Guide

Localization of a Mobile Robot

Author: Srikanth Nanda
Course of Studies: Business Intelligence and Data Analytics

First examiner: Prof. Dr. Elmar Wings

Second examiner: Prof. Dr. Jonas Schwarz

Contents

List of figures v

List of tables vii

Acronyms ix

1 Hints 1
- 1.1 Development Strategy . . . 1
- 1.2 Code Quality Recommendations . . . 1

2 Main Function 3
- 2.1 Software Architecture . . . 3
- 2.2 Core Modules . . . 3
- 2.3 Algorithm Pipeline . . . 4

3 Sketch 5

4 List Of Parts 7
- 4.1 Hardware Components - SparkFun Jetbot . . . 7
  - 4.1.1 Compute Module . . . 7
  - 4.1.2 Drive System . . . 7
  - 4.1.3 Sensor Package . . . 7
  - 4.1.4 Power Distribution . . . 8
  - 4.1.5 Chassis and Mechanical . . . 8
- 4.2 Software Dependencies . . . 8
  - 4.2.1 Vision and Image Processing . . . 8
  - 4.2.2 Pose Graph Optimization . . . 8
  - 4.2.3 Linear Algebra . . . 8
  - 4.2.4 Middleware and ROS . . . 9
  - 4.2.5 GPU Acceleration . . . 9
- 4.3 Development Tools . . . 9
  - 4.3.1 Compilers and Build System . . . 9
  - 4.3.2 Version Control and Collaboration . . . 9
  - 4.3.3 Debugging and Profiling . . . 9
  - 4.3.4 Visualization . . . 10
- 4.4 Testing and Validation . . . 10
  - 4.4.1 Benchmarking . . . 10
  - 4.4.2 Test Datasets . . . 11

4.4.3 Validation Scripts 11

5 Set Up **13**
5.1 SparkFun Jetbot Hardware Assembly 13
5.1.1 Component Verification Before Assembly 13
5.1.2 Critical GPIO Pinouts for Motor Control 13
5.1.3 Motor Driver H-Bridge Configuration 13
5.2 Jetson Nano Development Environment Setup 14
5.2.1 Operating System Foundation 14
5.2.2 Build Tools and Compilers 14
5.2.3 Critical Libraries for SparkFun Jetbot 15
5.2.4 Building on Jetson Nano Hardware Constraints 15
5.3 Encoder and Motor Hardware Interface 15
5.3.1 Quadrature Encoder Reading Specifications 15
5.3.2 PWM Motor Control Output 16

6 Functions **17**
6.1 API Documentation 17
6.1.1 CameraModule::captureFrame() 17
6.1.2 FeatureExtractor::extractKeypoints() 17
6.1.3 OdometryEstimator::computeTransform() 17
6.1.4 SLAMBackend::optimizePoseGraph() 17
6.2 Thread Management 17

7 Maintenance **19**
7.1 Code Repository Maintenance 19
7.1.1 Branch Strategy 19
7.1.2 Contribution Workflow 19
7.2 Documentation Updates 20
7.3 Performance Monitoring 20
7.4 Dependency Management 20

8 Safety Guidelines **21**
8.1 Development Environment Safety 21
8.2 Testing Safety Practices 21
8.3 Code Review Standards 21
8.4 Performance and Reliability 21

9 Troubleshooting **23**
9.1 Debugging Tools and Techniques 23
9.1.1 Logging and Visualization 23
9.1.2 Common Algorithmic Issues 23
9.2 Performance Debugging 24
9.3 Test Regression Detection 24

10 Add Ons **25**
10.1 Extending the Localization Framework 25
10.1.1 Adding New Feature Extractors 25
10.1.2 Integrating Alternative SLAM Backends 25
10.1.3 Adding New Sensor Modalities 25
10.2 Advanced Topics 25
10.2.1 Switching Between Active Feature Extractors 25
10.2.2 Collaborative Localization (Multi-Robot) 26
10.2.3 Real-Time Scheduling and Priority 26

List of Figures

2.1 Core software modules and their data flow within the localization system architecture 3
2.2 Six-stage localization algorithm pipeline, from raw sensor acquisition to map and loop-closure management 4

3.1 Monocular visual localization sketch showing camera field of view, detected features, furniture, and the traversable boundary 5

4.1 Iterative performance monitoring cycle covering profiling, resource tracking, timing, and trajectory error comparison 10

5.1 Signal flow between the Jetson Nano GPIO header, motor driver H-bridge, and DC motors with quadrature encoders 13

6.1 Real-time thread scheduling priorities for camera capture, feature extraction, pose estimation, and pose graph optimization 18

7.1 Git branching strategy showing the relationship between master, develop, and feature branches 19
7.2 Five-step contribution workflow for repository maintenance, from branch creation to merge approval 20

9.1 Decision tree for diagnosing common algorithmic issues in feature detection, matching, and pose estimation 24

List of Tables

5.1 Jetson Nano GPIO pinout for SparkFun Jetbot motor and encoder interfacing 14

Acronyms

1 Hints

1.1 Development Strategy

- Use branches for experimental features; maintain stable master branch
- Implement unit tests for feature extraction and matching algorithms
- Profile performance-critical sections (feature matching, bundle adjustment)
- Log intermediate results for algorithm debugging and offline analysis
- Version control all calibration parameters and configuration files
- Document algorithm parameters and their sensitivity to tuning

1.2 Code Quality Recommendations

- Follow consistent naming conventions across all modules
- Separate algorithm logic from I/O operations for testability
- Maintain comprehensive documentation for algorithm choices and trade-offs
- Use configuration files rather than hard-coded parameters
- Implement error handling for sensor timeouts and frame drops
- Provide visualization tools for debugging trajectory estimates

2 Main Function

2.1 Software Architecture

The localization system is built on a modular architecture with clearly separated concerns: sensor interfacing, feature extraction, algorithm execution, and result analysis.

2.2 Core Modules

- **Vision Module**: Processes camera frames, extracts features (SIFT, ORB, etc.)
- **Odometry Module**: Computes relative motion from encoders and visual features
- **SLAM Engine**: Performs simultaneous localization and map building
- **Trajectory Manager**: Maintains position history and path records
- **Sensor Fusion**: Integrates multiple sensor streams for robust estimation

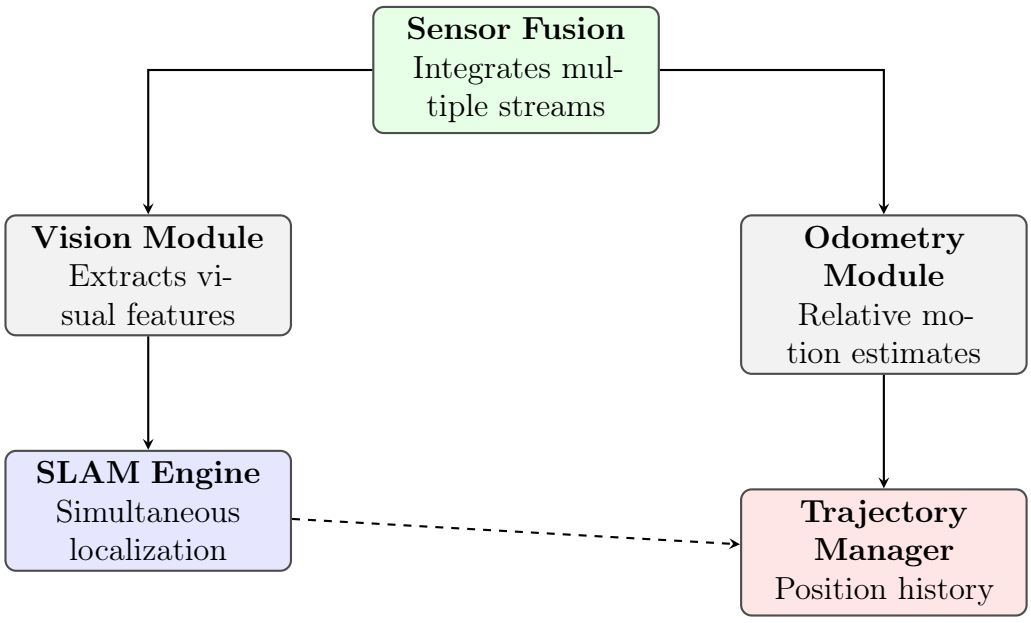


Figure 2.1: Core software modules and their data flow within the localization system architecture

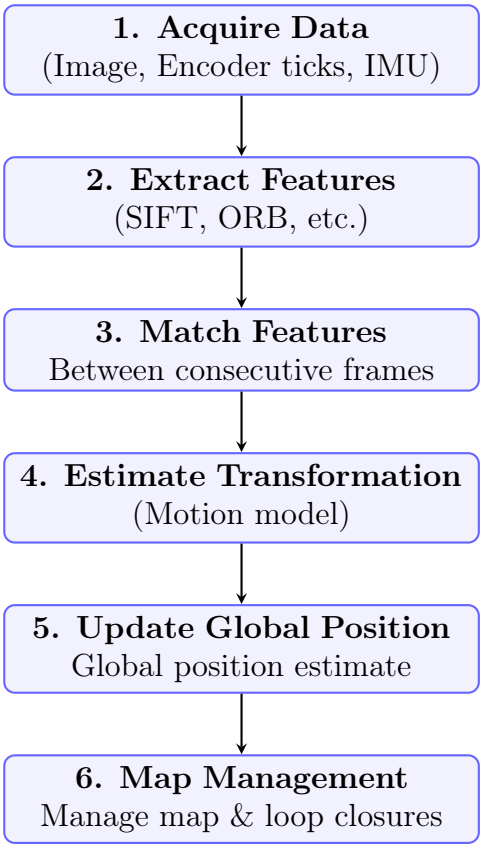


Figure 2.2: Six-stage localization algorithm pipeline, from raw sensor acquisition to map and loop-closure management

2.3 Algorithm Pipeline

1. Acquire raw sensor data (image, encoder ticks, optional IMU)
2. Extract and describe visual features
3. Match features between consecutive frames
4. Estimate transformation (motion model)
5. Update global position estimate
6. Manage map representation and loop closures

3 Sketch

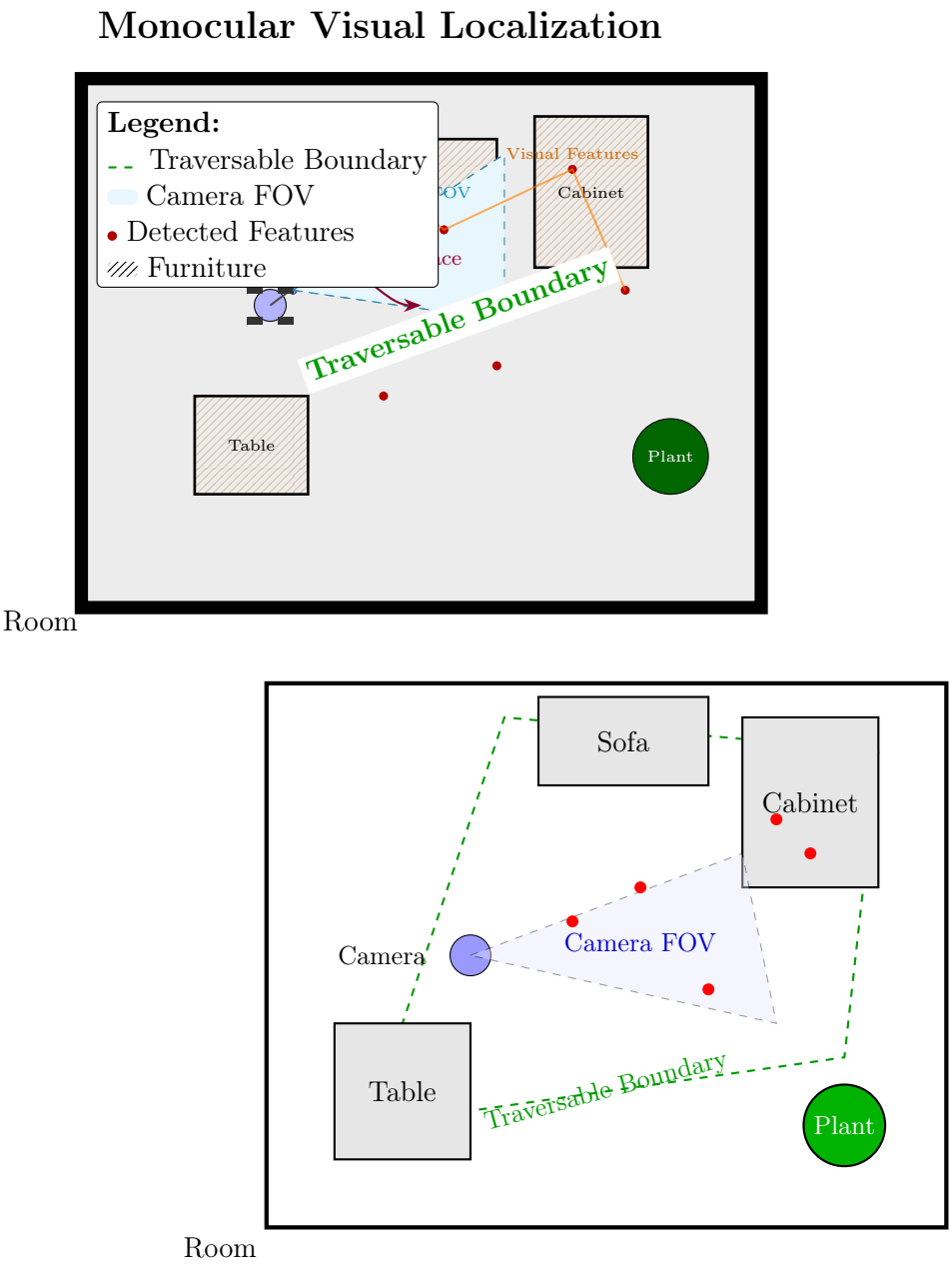
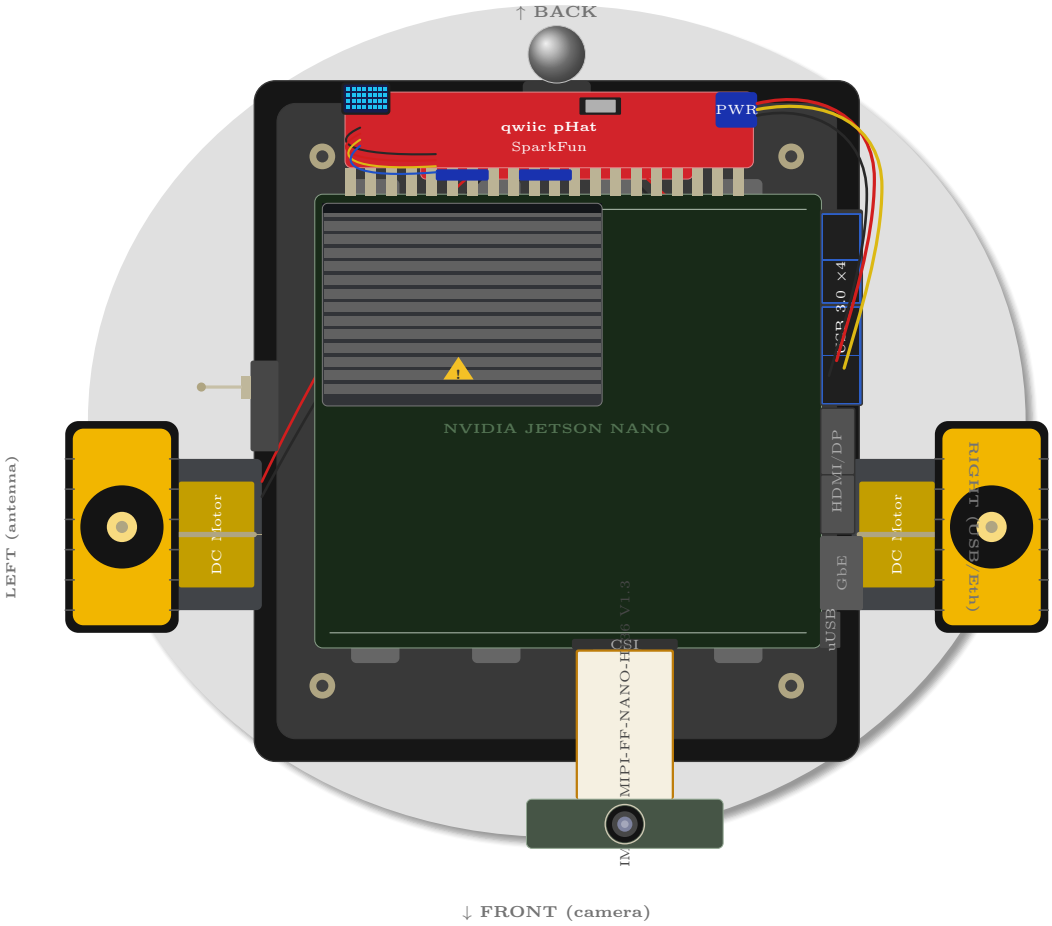


Figure 3.1: Monocular visual localization sketch showing camera field of view, detected features, furniture, and the traversable boundary



SparkFun JetBot AI Kit — Top View

4 List Of Parts

4.1 Hardware Components - SparkFun Jetbot

4.1.1 Compute Module

- **NVIDIA Jetson Nano V2**: 128 CUDA cores, 4GB LPDDR4, 5W-10W power
- **Storage**: microSD card (64GB minimum recommended, UHS-I compatible)
- **Power Input**: USB-C (5V/3A) or barrel connector (5V recommended)

4.1.2 Drive System

- **Motors**: 2x TT-motors (12V DC nominal, 100 RPM at rated)
- **Motor Current Draw**: Approximately 250mA per motor at rated load
- **Encoders**: Dual quadrature encoders (48 PPR, 7.5° resolution)
- **Encoder Frequency**: Up to 2.56 kHz per motor at maximum speed
- **Motor Driver**: L298N or DRV8833 H-Bridge (2A rated minimum)
- **Gearing Ratio**: 32:1 reduction (motor shaft to wheel)
- **Wheels**: 66mm diameter (practical 10mm ground clearance)
- **Track Width**: 90mm center-to-center

4.1.3 Sensor Package

- **Camera**: USB UVC-compliant webcam (640x480 @ 30fps typical)
- **Camera FOV**: Approximately 90° (varies by lens)
- **USB Hub**: 4-port hub recommended for camera + future sensors
- **Optional IMU**: Qwiic I2C connector for 9-DOF IMU (LSM9DS1)
- **Optional Lidar**: 2D or 3D Lidar via USB or GPIO Serial

4.1.4 Power Distribution

- **Battery Option 1**: 4xAA alkaline/rechargeable (6V nominal)
- **Battery Option 2**: USB power adapter (5V/3A minimum)
- **Motor Supply**: Separate 5V rail from Jetson power
- **Current Budget**: Jetson (5W) + Motors (0.5A each) + Camera (0.5A) = 2A total

4.1.5 Chassis and Mechanical

- **Chassis**: SparkFun JetBot robot platform (SKU ROB-16417)
- **Mounting Hardware**: M3 screws, standoffs for Jetson mounting
- **Cable Management**: USB cables, GPIO jumper wires
- **Protective Case**: Optional 3D-printed bumpers or acrylic chassis

4.2 Software Dependencies

4.2.1 Vision and Image Processing

- **OpenCV 3.3+**: Core library for feature detection, matching, calibration
- **Algorithms Included**: ORB (Oriented FAST and Rotated BRIEF) detector
- **Installation**: Pre-compiled for Jetson via `libopencv-dev`

4.2.2 Pose Graph Optimization

- **g2o**: General framework for optimizing graph-based SLAM
- **Build Time**: 15-20 minutes on Jetson Nano with -j2
- **Dependencies**: Eigen 3.x (linear algebra), Cholmod (sparse matrix)
- **CUDA Acceleration**: Maxwell GPU support (-DCUDA_ARCH_BIN=5.2)

4.2.3 Linear Algebra

- **Eigen 3.x**: Header-only library for matrix operations
- **SuiteSparse**: Optional sparse matrix optimization
- **LAPACK/BLAS**: Underlying numerical compute (via system packages)

4.2.4 Middleware and ROS

- **ROS Melodic or Noetic**: Message passing and module coordination
- **catkin**: Build system and dependency management
- **python-catkin-tools**: Alternative build front-end
- **rospy/roscpp**: Python and C++ client libraries

4.2.5 GPU Acceleration

- **CUDA 10.2**: GPU compute toolkit (pre-installed on JetPack)
- **cuDNN**: Optimized primitives for deep learning (optional)
- **TensorRT**: Model optimization for inference (optional)

4.3 Development Tools

4.3.1 Compilers and Build System

- **GCC 7+**: ARMv8 compiler with NEON SIMD support
- **CMake 3.10+**: Cross-platform build configuration
- **Make**: Parallel job execution (limit to -j2 on Jetson)
- **Autotools**: Legacy project support

4.3.2 Version Control and Collaboration

- **Git**: Distributed version control system
- **GitHub/GitLab**: Remote repository hosting
- **SSH Keys**: For secure GitHub access

4.3.3 Debugging and Profiling

- **gdb**: GNU debugger for step-through debugging
- **Valgrind**: Memory profiler and leak detector
- **perf**: Linux performance analysis tools
- **ROS tools**: rosbag, rostopic, rosnode utilities

4.3.4 Visualization

- **RViz**: 3D visualization tool for poses and maps
- **Python matplotlib**: 2D trajectory and error plotting
- **OpenCV imshow**: Real-time feature and match visualization

4.4 Testing and Validation

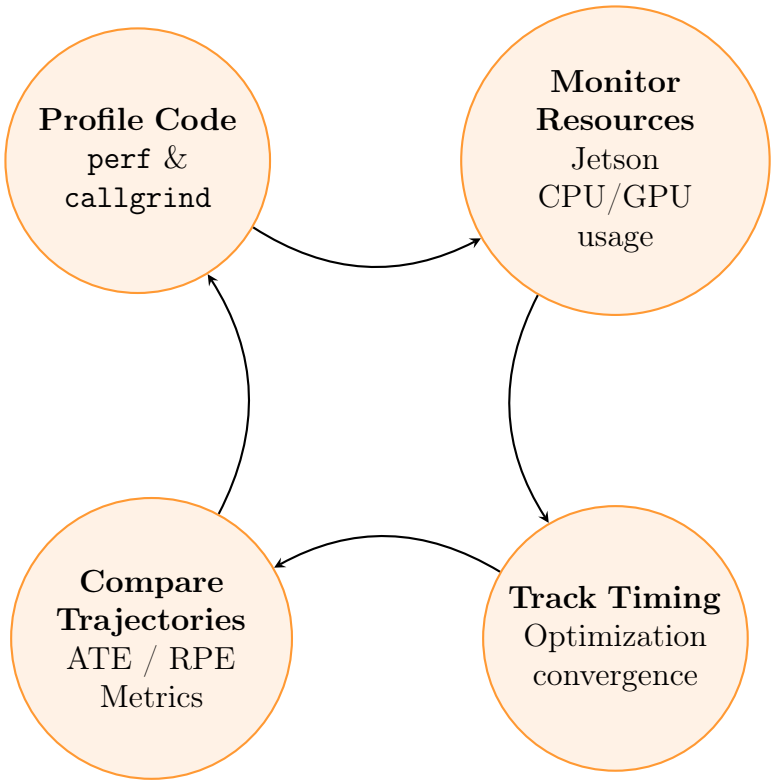


Figure 4.1: Iterative performance monitoring cycle covering profiling, resource tracking, timing, and trajectory error comparison

4.4.1 Benchmarking

- **Absolute Trajectory Error (ATE)**: Metric for global accuracy
- **Relative Pose Error (RPE)**: Metric for local consistency
- **Feature Detection Speed**: frames/second benchmark
- **Optimization Timing**: g2o convergence and timing analysis

4.4.2 Test Datasets

- **Indoor Sequences**: Custom recordings from SparkFun Jetbot
- **Standard Benchmarks**: TUM, KITTI dataset compatibility (optional)
- **Synthetic Data**: OpenGL-rendered sequences for algorithm testing

4.4.3 Validation Scripts

- **Python Stack**: numpy, scipy, pandas for data analysis
- **Evaluation Utilities**: Scripts computing ATE/RPE metrics
- **Plotting Tools**: Trajectory visualization and comparison

5 Set Up

5.1 SparkFun Jetbot Hardware Assembly

5.1.1 Component Verification Before Assembly

- **Jetson Nano Developer Kit V2**: Verify "v2" marking on board silkscreen
- **SparkFun Chassis**: JetBot robot platform (SKU ROB-16417 or newer)
- **DC Motors**: 2x TT-motors pre-installed in chassis (3-6V rated)
- **Quadrature Encoders**: 2x encoders at motor shafts (48 PPR standard)
- **USB Camera**: Standard webcam with UVC compatibility (640x480 typical)
- **Power Distribution**: 4xAA battery holder or 5V/3A USB-C adapter minimum

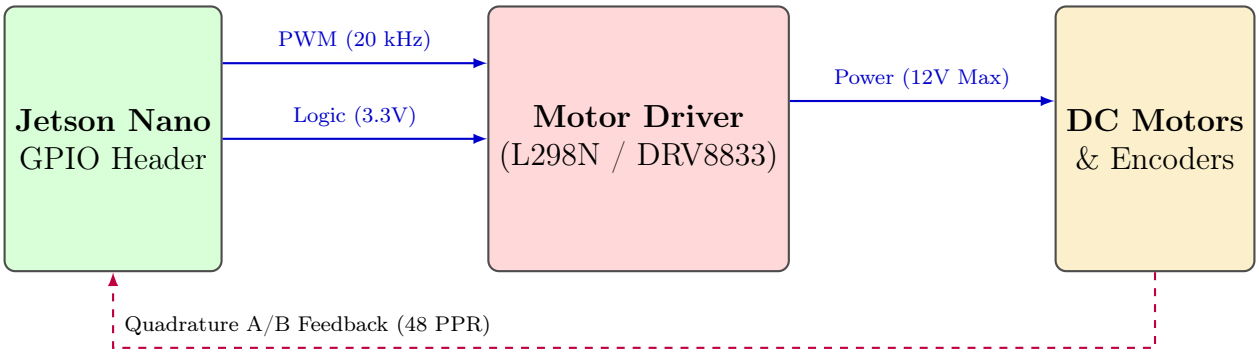


Figure 5.1: Signal flow between the Jetson Nano GPIO header, motor driver H-bridge, and DC motors with quadrature encoders

5.1.2 Critical GPIO Pinouts for Motor Control

Motor encoder and PWM connections to Jetson Nano GPIO Header:

5.1.3 Motor Driver H-Bridge Configuration

Required motor control electronics:

Signal	GPIO	Pin	Description
Motor L Encoder A	GPIO33	15	Phase A, rising edge forward
Motor L Encoder B	GPIO88	16	Phase B, 90° offset
Motor R Encoder A	GPIO56	37	Phase A, rising edge forward
Motor R Encoder B	GPIO90	38	Phase B, 90° offset
Motor L PWM	GPIO79	18	PWM output, 20 kHz nominal
Motor R PWM	GPIO80	32	PWM output, 20 kHz nominal
I2C Bus SDA	GPIO2	3	1.8V logic (optional Qwiic)
I2C Bus SCL	GPIO3	5	1.8V logic (optional Qwiic)

Table 5.1: Jetson Nano GPIO pinout for SparkFun Jetbot motor and encoder interfacing

- **Driver IC**: L298N or DRV8833 dual-channel motor driver recommended
- **Input Logic Voltage**: 3.3V signals from Jetson GPIO
- **Motor Supply Voltage**: 5V rail separate from Jetson power
- **Ground Reference**: Common ground between Jetson and motor driver
- **PWM Frequency**: 20 kHz (minimize switching losses and acoustic noise)
- **Dead Time**: Insert 1-2 microseconds between phase transitions
- **Current Limiting**: Per-motor limit approximately 500mA

5.2 Jetson Nano Development Environment Setup

5.2.1 Operating System Foundation

- JetPack 4.6.1 or newer with Ubuntu 18.04 LTS or 20.04 LTS (Arm64)
- Prepare 2GB minimum free disk space for development tools
- Execute system updates: `sudo apt-get update && sudo apt-get upgrade -y`

5.2.2 Build Tools and Compilers

Essential packages for compiling SparkFun Jetbot software:

- **Build Foundation**: GCC 7+ with ARMv8 architecture support
- **Build System**: CMake 3.10+, Make, autotools
- **Version Control**: Git with full functionality
- **Python**: Python 3.6+ with pip, numpy, opencv-python

5.2.3 Critical Libraries for SparkFun Jetbot

OpenCV 3.3+ Pre-compiled for Jetson available via `sudo apt install libopencv-dev`

g2o Optimizer Pose graph SLAM backend (compile from source, 15-20 minutes on Jetson)

Eigen 3.x Linear algebra library (dependency of g2o)

ROS Melodic/Noetic Full desktop environment with catkin build tools

CUDA 10.2 GPU acceleration toolkit (pre-installed on JetPack)

cuDNN Neural network library (optional, for deep learning)

5.2.4 Building on Jetson Nano Hardware Constraints

Memory and thermal considerations:

- **Parallel Build Jobs**: Use `make -j2` ONLY (Jetson has 4GB RAM)
- **SIMD Optimization**: Enable with `-march=armv8-a -mfpu=neon`
- **CUDA GPU Targeting**: Set `-DCUDA_ARCH_BIN=5.2` for Maxwell GPU
- **Alternative Strategy**: Cross-compile on x86 workstation for faster builds
- **Thermal Management**: Monitor CPU temperature during compilation

5.3 Encoder and Motor Hardware Interface

5.3.1 Quadrature Encoder Reading Specifications

- **Resolution**: 48 increments per complete wheel rotation (° per tick)
- **Phase Logic**: Channel A leads Channel B during forward motion
- **Maximum Frequency**: 2.56 kHz per encoder at rated motor RPM
- **Reading Implementation**: GPIO sysfs poller or kernel interrupt handler
- **Synchronization**: Timestamp all encoder reads with frame captures

5.3.2 PWM Motor Control Output

- **GPIO Pins**: 18 (GPIO79) for left motor, 32 (GPIO80) for right motor
- **Frequency**: 20 kHz via `/sys/class/pwm/pwmchip0`
- **Duty Cycle**: 0-100% represented as 0-100000 in sysfs
- **Ramp Rate**: Avoid abrupt transitions (limit to 100% per second maximum)
- **Direction Control**: Separate GPIO pairs for motor polarity selection

6 Functions

6.1 API Documentation

6.1.1 CameraModule::captureFrame()

Acquires a single frame from camera and returns as cv::Mat.

- Returns: BGR image, 640x480 resolution
- Throws: CameraException on timeout

6.1.2 FeatureExtractor::extractKeypoints()

Extracts and describes features from image.

- Parameters: image (cv::Mat), algorithm (string: `SIFT`/`ORB`)
- Returns: std::vector<KeyPoint>, cv::Mat descriptors

6.1.3 OdometryEstimator::computeTransform()

Estimates rigid transformation between consecutive frames.

- Parameters: prev_descriptors, curr_descriptors, matches
- Returns: cv::Mat transformation matrix (4x4)

6.1.4 SLAMBackend::optimizePoseGraph()

Performs pose graph optimization using g2o.

- Parameters: pose_graph, constraints
- Returns: optimized poses, final error

6.2 Thread Management

Main threads:

- Camera capture thread (real-time priority)

- Feature extraction thread
- Pose estimation thread
- Optimization thread (lower priority)

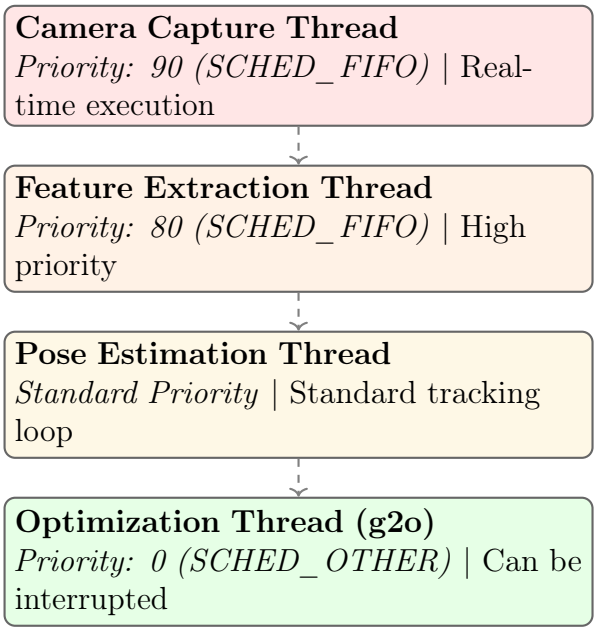


Figure 6.1: Real-time thread scheduling priorities for camera capture, feature extraction, pose estimation, and pose graph optimization

7 Maintenance

7.1 Code Repository Maintenance

7.1.1 Branch Strategy

- `master`: Stable release code, tested and documented
- `develop`: Integration branch for completed features
- `feature/*`: Individual feature development

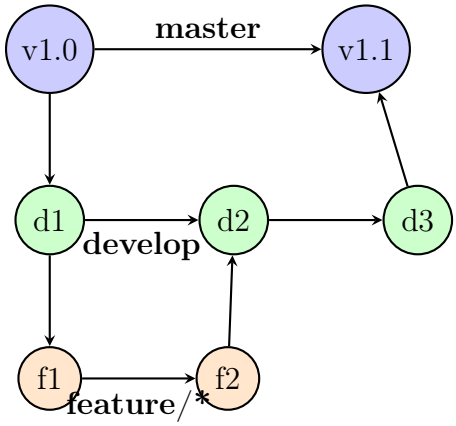


Figure 7.1: Git branching strategy showing the relationship between master, develop, and feature branches

7.1.2 Contribution Workflow

1. Create feature branch from develop
2. Implement and test locally
3. Submit pull request with test results
4. Address code review comments
5. Merge to develop after approval

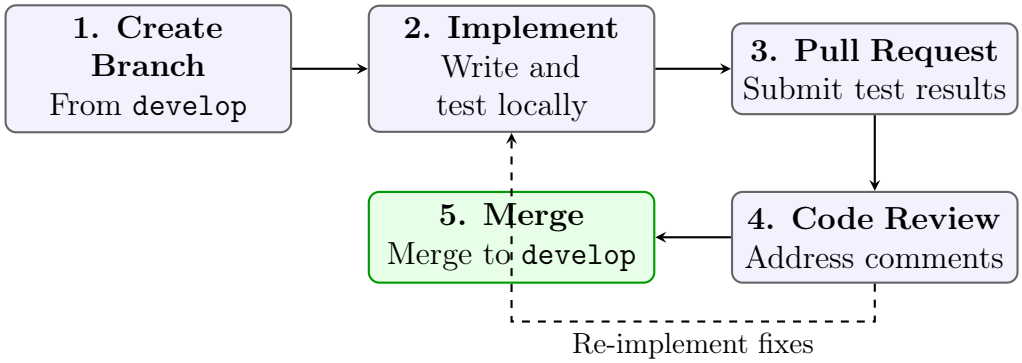


Figure 7.2: Five-step contribution workflow for repository maintenance, from branch creation to merge approval

7.2 Documentation Updates

- Update API documentation when modifying class signatures
- Maintain changelog (CHANGELOG.md) with all modifications
- Document algorithm parameter tuning experiences
- Keep config file examples synchronized with code

7.3 Performance Monitoring

- Profile code after major algorithm changes
- Compare frame processing times on reference trajectories
- Monitor memory usage over long runs (check for leaks)
- Track pose optimization convergence times

7.4 Dependency Management

- Keep OpenCV, g2o, and ROS packages updated
- Document minimum version requirements
- Test compatibility after system updates

8 Safety Guidelines

8.1 Development Environment Safety

- Never modify core localization algorithms without testing in simulator first
- Always implement emergency stop handlers in new code
- Use feature flags (configuration options) rather than code comments for experimental features
- Include error handling and graceful degradation for sensor failures

8.2 Testing Safety Practices

- Create unit tests that verify safety guarantees (e.g., max acceleration limits)
- Test error recovery paths, not just happy paths
- Simulate sensor failures and verify appropriate robot response
- Use safety limits (max speed, max turn rate) in all motion commands

8.3 Code Review Standards

- All changes to motion control require at least two approvals
- Safety-critical sections must have comprehensive comments
- Require evidence of both unit testing and hardware testing
- Document any relaxing of safety constraints and provide justification

8.4 Performance and Reliability

- Monitor real-time thread priorities; ensure safety loops run at highest priority
- Implement watchdog timers to detect system hangs
- Test code under high CPU load to verify safety features remain responsive

- Use mutex and spinlock carefully to avoid deadlocks that could leave motors running

9 Troubleshooting

9.1 Debugging Tools and Techniques

9.1.1 Logging and Visualization

- Enable debug logging: `export ROS_LOG_LEVEL=debug`
- Visualize pose graph: `rosrun g2o viewer`
- Plot trajectories with Python scripts in `scripts/`
- Use `rqt_graph` to verify message flow

9.1.2 Common Algorithmic Issues

- **Features not detected**: Check image histogram; may need exposure adjustment
- **Matches appear random**: Feature descriptor mismatch; verify matcher threshold
- **Pose estimates explode**: Check for numerical instability in transformation estimation
- **Bundle adjustment diverges**: Initial guess too far from solution; relax convergence tolerance initially

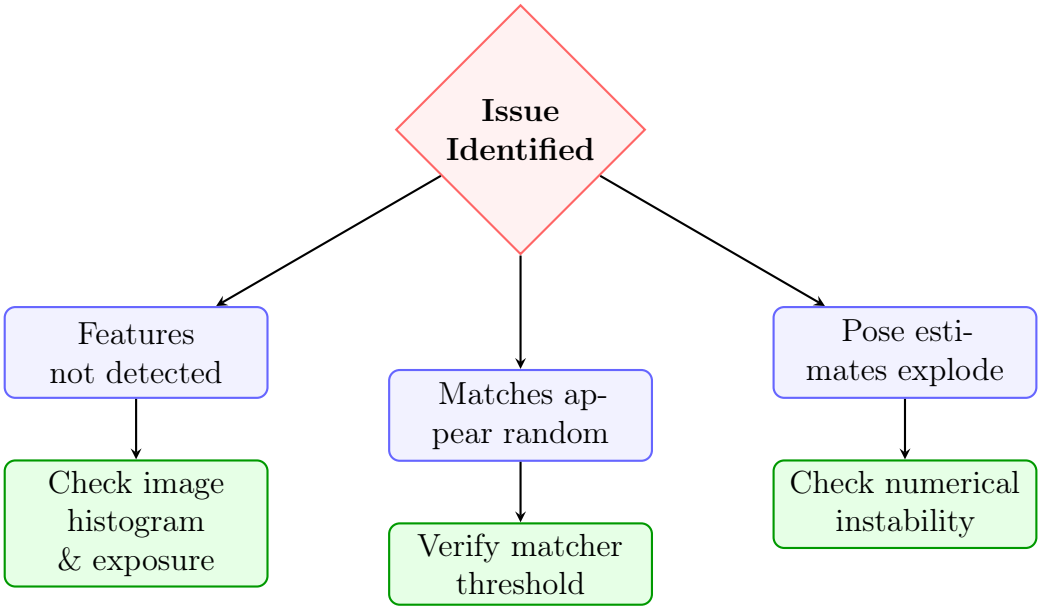


Figure 9.1: Decision tree for diagnosing common algorithmic issues in feature detection, matching, and pose estimation

9.2 Performance Debugging

- Profile with `perf` and `callgrind` to identify bottlenecks
- Use `top` to monitor Jetson CPU/GPU/memory during runs
- Check TensorRT optimization if using GPU-accelerated feature extraction
- Consider OpenCV CUDA support for real-time performance

9.3 Test Regression Detection

- Maintain benchmark trajectories in `test/data/`
- Run `./test/run_benchmarks.sh` after major changes
- Compare Absolute Trajectory Error against baseline
- Document acceptable regression thresholds

10 Add Ons

10.1 Extending the Localization Framework

10.1.1 Adding New Feature Extractors

1. Implement `FeatureExtractor` interface in `src/feature_extraction/`
2. Override `extractKeypoints()` and `computeDescriptors()`
3. Register in `FeatureFactory::create()` method
4. Add configuration option in `config/localization.yaml`

10.1.2 Integrating Alternative SLAM Backends

- Replace g2o with Ceres Solver, GTSAM, or iSAM2
- Implement `SLAMBackend` abstract interface
- Maintain pose graph input/output format compatibility
- Benchmark against baseline for validation

10.1.3 Adding New Sensor Modalities

1. Create new `SensorModule` derived class
2. Implement sensor-specific data acquisition (in separate thread)
3. Define constraint type (encoder, IMU, Lidar, Sonar)
4. Add to `SensorFusionEngine`

10.2 Advanced Topics

10.2.1 Switching Between Active Feature Extractors

- Load different extraction algorithms at runtime via config
- Useful for adaptive selection based on environment
- Example: Use SIFT in low-texture, ORB elsewhere for speed

10.2.2 Collaborative Localization (Multi-Robot)

- Share pose graph constraints across network using ROS topics
- Requires synchronization and outlier filtering
- Significantly improves global map accuracy
- See `examples/multi_robot_slam.cpp`

10.2.3 Real-Time Scheduling and Priority

- Camera capture thread: `SCHED_FIFO` priority 90
- Feature extraction: `SCHED_FIFO` priority 80
- Optimization: `SCHED_OTHER` priority 0 (can be interrupted)
- See `src/thread_manager.cpp` for priority configuration